

APIS AND FASTAPI

WEEK 2, DAY 1-2

What an API is. What FastAPI is, and why it is the default.
Then we build one.



PART ONE: WHAT IS AN API?

Before any code, any framework, any syntax

THE DEFINITION, IN ONE SENTENCE

EVERYTHING ELSE IS DETAIL

**A WAY FOR ONE PROGRAM
TO ASK ANOTHER PROGRAM
FOR SOMETHING.**

That is it. No network required, no internet required, no JSON required.
Those are all just common ways of doing it.

THE PART THAT ACTUALLY MATTERS

NOT THE ANALOGY, THE CONSEQUENCE

AN API IS A PROMISE, MADE TO A STRANGER

- | | | |
|----------|-----------------------|--|
| 1 | you ask | In a format they agreed to accept |
| 2 | they answer | In a format you agreed to expect |
| 3 | neither of you | Knows anything about the other's internals |

That last line is the whole value. You can rewrite everything behind an API and nobody calling it needs to know.

EVERY WEB REQUEST HAS FOUR PARTS

LEARN THESE AND THE REST IS VOCABULARY

Part	What it is
METHOD	The verb. GET to fetch, POST to create, DELETE to remove.
PATH	What you are asking about. /books, or /books/3
BODY	The data you are sending, if any. Usually JSON.
STATUS	The answer's headline. 200 fine, 404 not there, 422 bad request.

A request is a method plus a path. A response is a status plus a body. Everything today is those four things.

A STATUS CODE IS AN INSTRUCTION

NOT A REPORT ON THE SERVER

Code	What it tells the caller to do next
200 OK	Use the answer. It worked.
201 Created	The thing now exists. Here is where to find it.
404 Not Found	It is not there. Do not retry, it will not appear.
422 Unprocessable	Your request was malformed. Fix it, then retry.
500 Server Error	We broke. Not your fault. Retrying might work.

Pick the code by what the caller should do next, not by what happened on your side.

PART TWO: WHAT IS FASTAPI?

And why has it become the default

A PYTHON FRAMEWORK FOR BUILDING APIS

BUILT ON TWO THINGS, NOT FROM SCRATCH

STARLETTE

Handles the HTTP plumbing.
Receiving requests,
sending responses.

PYDANTIC

Handles data validation.
Checking what arrives
is what you expected.

FastAPI's own contribution: it reads your Python type hints and wires those two together automatically.

THE IDEA THE WHOLE FRAMEWORK RESTS ON

YOUR TYPE HINTS DO REAL WORK

YOU WRITE THIS

```
def get_book(book_id: int):
```

AND YOU GET, WITHOUT WRITING ANY OF IT

- The value pulled out of the URL and converted to an integer
- A 422 error, automatically, if somebody sends /books/abc
- Documentation saying this endpoint takes an integer

One annotation. Three jobs. That is the pitch.

WHY IT BECAME THE DEFAULT

THREE REASONS, IN ORDER OF HOW MUCH THEY SAVE YOU

WHAT YOU STOP WRITING BY HAND

- 1 validation** Malformed input is rejected before your code runs.
- 2 documentation** Generated from the code. It cannot drift out of date.
- 3 concurrency** Async-native. Built for waiting on slow things efficiently.

That third one is why it suits AI work specifically: most of an LLM request is spent waiting.

WHERE IT SITS AGAINST THE ALTERNATIVES

ALL THREE ARE STILL IN ACTIVE USE

Framework	Where it genuinely wins
FastAPI	API-first services, async work, anything waiting on a model or another service
Django	Large business systems needing an admin panel, auth and database tooling
Flask	Small internal tools and older codebases. Simple, but you write more yourself

Companies that standardised before 2020 mostly run Flask. Those choosing since 2021 mostly chose FastAPI.

WHY WE ARE USING IT ON THIS PROGRAMME

NOT BECAUSE IT IS FASHIONABLE

**EVERY AGENT, TOOL AND MODEL
YOU BUILD FROM WEEK 3 ONWARDS
NEEDS SOMETHING TO SIT BEHIND.**

Learning it now, before any model is involved, means Week 5 is about agents and Week 6 is about deployment, not about learning a framework under pressure.



PART THREE: WHAT WE ARE BUILDING

The code-along, and why each step is there

A FIRM INTELLIGENCE API

FIVE ENDPOINTS, BUILT ONE AT A TIME

Endpoint	What it does
GET /health	Says the service is alive. The simplest possible endpoint.
GET /firms	Returns every firm in the dataset.
GET /firms/{id}	Returns one firm, or a 404 if it does not exist.
GET /firms/{id}/benchmarks	Revenue per lawyer, profit per equity partner.
POST /firms	Adds a firm, and rejects bad data properly.

Synthetic data throughout. Every firm name and figure is invented for teaching.

THE SIX STEPS, AND WHY EACH ONE EXISTS

NOTHING HERE IS FILLER

- | | | |
|----------|-------------------------|---|
| 1 | Set up a venv | So this project's packages cannot break any other project |
| 2 | Install FastAPI | Two packages: the framework, and the server that runs it |
| 3 | First endpoint | Prove the whole loop works before adding anything |
| 4 | Add the dataset | Firm records, and an endpoint that returns them |
| 5 | One firm, or 404 | Your first real decision: what a missing thing means |
| 6 | Accept input | Validation rules that exist for a reason, not for form |

ONE RULE FOR THE NEXT THREE HOURS

IT IS THE ONLY ONE THAT MATTERS

Run it after every single change.

WHY

If you write four things and then run it, and it breaks, you have four suspects.

If you run it after each one, you always have exactly one.

AN API IS A PROMISE

Made to somebody you will never meet, who cannot ask you what you meant, and who only has what your response actually says.

For the next three hours, you are the one making those promises.

LAPTOPS OPEN. LET'S BUILD IT.